

Natural Language Processing: Notes

ML A-Z — Part 7

1 What this notebook implements

The notebook `natural_language_processing.ipynb` builds a sentiment classifier for restaurant reviews using a classic **Bag of Words** pipeline:

1. **Load the data.** Read `Restaurant_Reviews.tsv`, a tab-separated file of 1000 reviews, each labeled 0 (negative) or 1 (positive).
2. **Clean the text.** For each review:
 - strip everything except letters (`re.sub('[^a-zA-Z]', ' ', ...)`),
 - lowercase and tokenize by splitting on whitespace,
 - remove English stopwords (`nltk.corpus.stopwords`), *except* the word "not", which is kept because negation carries sentiment information,
 - stem each remaining token with the Porter stemmer (e.g. "loved" → "love").
3. **Vectorize.** Turn the cleaned corpus into a Bag-of-Words matrix with `CountVectorizer(max_features=1500)` each review becomes a row, each of the 1500 most frequent remaining words becomes a column, and each entry is that word's count in that review.
4. **Split.** 80/20 train/test split (`random_state=0`).
5. **Classify.** Train a `GaussianNB` (Gaussian Naive Bayes) classifier on the training matrix.
6. **Evaluate.** Predict on the test set and report a confusion matrix and accuracy score.

1.1 Result

On the held-out test set (200 reviews), the model achieves:

$$\text{Confusion matrix} = \begin{pmatrix} 55 & 42 \\ 12 & 91 \end{pmatrix}, \quad \text{Accuracy} = 0.73$$

i.e. 55 true negatives, 91 true positives, 42 false positives, and 12 false negatives, for 73% overall accuracy.

2 About the delimiter

The dataset is loaded with:

```
dataset = pd.read_csv('Restaurant_Reviews.tsv',  
                      delimiter = '\t', quoting = 3)
```

2.1 `delimiter = '\t'`

`Restaurant_Reviews.tsv` is a **TSV** (tab-separated values) file, not a comma-separated CSV. Review text routinely contains commas (e.g. “Great food, terrible service”), so a comma delimiter would incorrectly split a single review into multiple fields. Using `'\t'` (the tab character) as the field separator avoids this, since tabs essentially never appear inside the review text itself.

2.2 `quoting = 3`

`quoting = 3` corresponds to `csv.QUOTE_NONE` in Python’s `csv` module. It tells the parser to treat the quote character (`"`) as an ordinary character instead of a field delimiter. This matters here because review text frequently contains stray double quotes (e.g. someone quoting the menu, or writing `"amazing"` sarcastically). With the default quoting behavior, pandas would try to interpret those quotes as marking the start/end of a quoted field, which can corrupt parsing (merging rows or truncating fields). Disabling quote interpretation makes the tab character the only thing that matters for splitting columns, which is exactly what a raw TSV of free-text reviews needs.

The four valid values of the `quoting` parameter (from Python’s `csv` module) are:

Value	Constant	Meaning
0	<code>QUOTE_MINIMAL</code>	Quote only fields containing special characters (default)
1	<code>QUOTE_ALL</code>	Quote every field
2	<code>QUOTE_NONNUMERIC</code>	Quote all non-numeric fields
3	<code>QUOTE_NONE</code>	Never treat the quote character specially